Projekt „SIMS“ („Security Incident Management System“)

System zum Protokollieren von IT-Sicherheitsvorfällen

Features

- Manuelle Erfassung von sicherheitsrelevanten Vorfällen (Bearbeiter, Melder, Schweregrad, Status, CVE, System, Beschreibung, Zeitstempel, etc.)
- Eskalation (z.B.: an nächsten Benutzer, oder irgendein „Level“ oder ähnliches)
- Benutzerverwaltung mit Rollen wie bspw. Administratoren, Benutzer
- Logging (SQL oder Redis): Id, Timestamp, Loglevel, Message
- Funktion: Benutzeranmeldung, Vorfall add/edit/close; Eskalation; User add/disable; Liste Incidents und User

Technische Vorgaben

Ziel: **mind. 4 Docker Container** in sinnvollen Docker-Networks (siehe Architektur-Grafik): App, APIs, Redis, SQLServer siehe Architektur auf **Seite 2**!

- Gesamte Verwaltung des Projekts mit GIT auf https://git.nwt.fhstp.ac.at oder Github
- Vermeiden von Code-Smells!
- **OOP-Programmierung**:
  - Komplett „fertiges“ lauffähiges Programm inkl. Speichern/Ändern/Lesen aller Daten in Datenbanken
  - Speichern der Daten durch Architekturpattern für Datenquelle (05-DB-AppDev.pdf) **oder** durch Entity-Framework
  - Vermeiden von SQL-Injections
- **APIs (RESTFUL)** (beispielsweise zur Userverwaltung und Authentifizierung oder als reichhaltiger „Core“ Service)
  - Deployment: Docker Image (Anwendung soll lokal komplett in Docker laufen)
- **REDIS NoSQL Datenbank**: Z.B.: Speichern Session (Username, Passwort-Hash) oder des Logs
- **SQL-Datenbank**: ER-Diagramm und Erstellung einer relationalen Datenbank mit SQL im Docker-Container
- **Docker**: Erstellen sie alle nötigen Dateien (Dockerfile) um die Anwendung komplett in Docker laufen zu lassen
- **Frontend**: Web-Frontend (LLM-generieriert). Minimal wäre ein Konsolen-Client
- **SAST**: Durchführen einer statischen Code-Analyse mit Semgrep (siehe Seite 2) oder alternativen Produkten
- **Security**: Dokumentieren (in README.md) an welche Security-Maßnahmen ihr gedacht- bzw. auch umgesetzt habt

Abgabe

- Link zum Github oder Gitlab-Repo von einer Person des Teams per eCampus
- SQL-Skript für Datenbankmodell (wenn nicht Entity-Framework)
- Dockerfiles (optional `docker-compose.yml`), keine Docker Images abgeben! 😉
- Dokumentation in Markdown als README.md
  - Badges in Markdown verwenden: https://badgen.net/
  - Projektaufgabe NICHT dokumentieren!
  - Ausführliche Dokumentation der Programmfunktionalität und der Features
  - Angabe einer Version, Lizenz, Mitwirkende, und (fiktiver) - Roadmap
  - Systemvoraussetzungen (OS, .NET-Runtime)
  - Link zu GIT-Repo
  - ER-Diagramm, Klassendiagramm (wenn sinnvoll)
  - Ergebnisse zu SAST (Semgrep) → siehe Seite 2

➔ Weitere Infos auf Seite 2!!!!!!!

Punkte (8-10)

- 1…3 Punkte Programmierung Microservices (1 Punkt pro sinnvollen MS, max. 3 MS, min. 1 MS)
- 3 Punkte Datenspeicherung (Patterns oder EF, SQL, NoSQL)
- 1 Punkt SAST und allgemeine Code-Qualität
- 1 Punkt Docker
- 1 Punkt Dokumentation (inkl. Security Maßnahmen und Überlegungen)
- 1 Punkt Funktionierende Demo

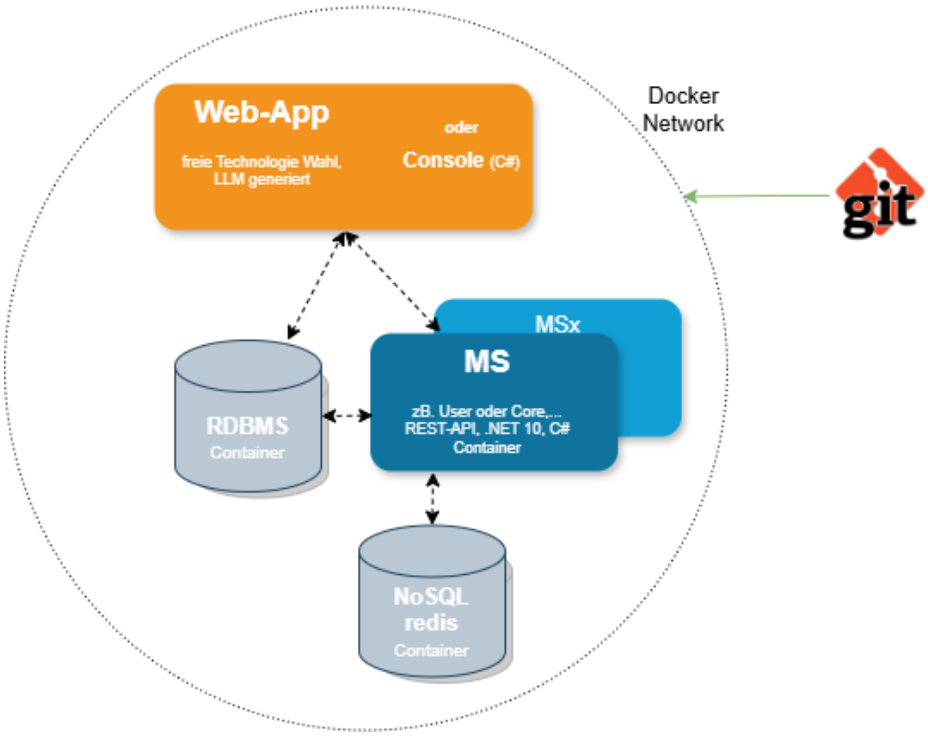
- **8-10 Punkte**

Sonderpunkte:

- Erstellung SBOM + DependencyTrack im Container
- CI-Pipeline (optional CD)
- Anbindung externer Systeme für die Erstellung von Vorfällen (für z.B.: Jira)
- Besondere „Sonderleistungen“ können ebenfalls zu Sonderpunkten führen 😊

Beispiel Architektur

Dies ist nur ein Beispiel als Anhaltspunkt. Bitte selbst überlegen, wie das Projekt gestaltet wird.



Tipp

SAST mit Semgrep:

Gelb markierten Pfad anpassen!

```
docker run --rm -v "c:/temp/MeinProjekt:/src" returntocorp/semgrep semgrep --config auto
```

SBOM CycloneDX:

```
dotnet tool install --global CycloneDX
dotnet-CycloneDX YourSolution.sln -o /output/path
```

Tipp zu CORS in der API:

```
builder.Services.AddCors(options =>
{
  options.AddPolicy("MyApp", policy =>
  {
    policy.WithOrigins("https://localhost:1234")
      .AllowAnyHeader()
      .AllowAnyMethod();
  });
});
[...]
//vor app.UseAuthorization() muss Cors aktiviert werden
app.UseCors("MyApp");
```